

Report: Assignment 2 (DA3408)

Question 1

1. Disk usage of spam-naive: 689 MB, Content size of spam-naive: 157 MB

```
ananya-kishore@ananya:~/da3408-assignment2/question_1$ docker images spam-naive
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
spam-naive:latest	0f50fa165d97	689MB	157MB	

2. Disk usage of spam-multistage:latest: 594 MB, Content size of spam-multistage:latest: 137 MB

```
ananya-kishore@ananya:~/da3408-assignment2/question_1$ docker images spam-multistage:latest
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
spam-multistage:latest	1aff2821a552	594MB	137MB	

Percentage reduction in image size (i.e, % reduction in disk usage) = $95/689 * 100 = 13.788\%$

3. The multi-stage image is smaller because the final python:3.10-slim runtime stage contains only **requirements-serve.txt**, **main.py** and the trained **model.joblib**. The training files (**generate_dataset.py**, **train.py**, **requirements-train.txt**) and their installed training dependencies (**pandas**, **scikit-learn**, **joblib**) remain in the builder stage and are not copied into the final image. The builder stage is therefore discarded from the final image, leaving only the runtime environment and files required to serve the API, reducing the disk usage from 689 MB to 594 MB (or, 13.788% size reduction).

Question 2

1. At question_2/main.py in the repo
2. At question_2/docker-compose.yml in the repo
3. Evidence:

```
ananya-kishore@ananya:~/da3408-assignment2/question_2$ curl http://localhost:5000/healthz
{"status": "ok"}
ananya-kishore@ananya:~/da3408-assignment2/question_2$ python3 evidence_script.py
Calling API with text: WIN a FREE iPhone now! Click here: bit.ly/xyz123
Call 1: 8.86 ms -> {'label': 'spam', 'cache': 'miss'}
Call 2: 4.49 ms -> {'label': 'spam', 'cache': 'hit'}
Call 3: 4.54 ms -> {'label': 'spam', 'cache': 'hit'}
Call 4: 4.79 ms -> {'label': 'spam', 'cache': 'hit'}
Call 5: 5.11 ms -> {'label': 'spam', 'cache': 'hit'}

Calling API with text: Congrats! So happy for you!
Call 1: 7.10 ms -> {'label': 'ham', 'cache': 'miss'}
Call 2: 4.90 ms -> {'label': 'ham', 'cache': 'hit'}
Call 3: 5.76 ms -> {'label': 'ham', 'cache': 'hit'}
Call 4: 4.63 ms -> {'label': 'ham', 'cache': 'hit'}
Call 5: 4.74 ms -> {'label': 'ham', 'cache': 'hit'}
```

```
ananya-kishore@ananya:~/da3408-assignment2/question_2$ time curl -s -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"text": "Hey, you free?"}'
{"label": "ham", "cache": "miss"}
real    0m0.010s
user    0m0.002s
sys      0m0.009s
ananya-kishore@ananya:~/da3408-assignment2/question_2$ time curl -s -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"text": "Hey, you free?"}'
{"label": "ham", "cache": "hit"}
real    0m0.014s
user    0m0.004s
sys      0m0.007s
```

4. **Docker Compose** solves the problem of defining, networking and running multiple related containers together on a single host, which cannot be done by a single Dockerfile. **Kubernetes** provides container orientation at the cluster scale, managing containers across multiple nodes with features such as scheduling, scaling, self-healing, and rolling updates.

Question 3

1. The cluster has 2 nodes with 2 CPUs each, giving **4 CPUs in total**. Each validation pod requests and is limited to 1 CPU. Hence, **at most $4/1 = 4$ pods can run at once**. Thus, I choose the **parallelism as 4** for the 8 completions.
2. Evidence of parallelism = 4:

```
ananya-kishore@ananya:~/da3408-assignment2/question_3$ kubectl get pods -o wide -w
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS GATES
signup-shard-validation-0-n8wp2	0/1	Completed	0	14s	10.244.1.21	minikube-m02	<none>	<none>	<none>
signup-shard-validation-1-8omnn	0/1	Completed	0	14s	10.244.1.22	minikube-m02	<none>	<none>	<none>
signup-shard-validation-2-zcs85	0/1	Completed	0	14s	10.244.0.23	minikube	<none>	<none>	<none>
signup-shard-validation-3-zhtgn	0/1	Completed	0	14s	10.244.0.24	minikube	<none>	<none>	<none>
signup-shard-validation-4-fjns5	0/1	ContainerCreating	0	1s	<none>	minikube-m02	<none>	<none>	<none>
signup-shard-validation-5-7mcbv	0/1	ContainerCreating	0	1s	<none>	minikube-m02	<none>	<none>	<none>
signup-shard-validation-6-42hdv	0/1	ContainerCreating	0	1s	<none>	minikube	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	0/1	ContainerCreating	0	1s	<none>	minikube	<none>	<none>	<none>
signup-shard-validation-6-42hdv	1/1	Running	0	1s	10.244.0.25	minikube	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	1/1	Running	0	1s	10.244.0.26	minikube	<none>	<none>	<none>
signup-shard-validation-4-fjns5	1/1	Running	0	1s	10.244.1.23	minikube-m02	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	0/1	Completed	0	1s	10.244.1.24	minikube-m02	<none>	<none>	<none>
signup-shard-validation-6-42hdv	0/1	Completed	0	11s	10.244.0.25	minikube	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	0/1	Completed	0	11s	10.244.0.26	minikube	<none>	<none>	<none>
signup-shard-validation-5-7mcbv	0/1	Completed	0	11s	10.244.1.24	minikube-m02	<none>	<none>	<none>
signup-shard-validation-4-fjns5	0/1	Completed	0	11s	10.244.1.23	minikube-m02	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	0/1	Completed	0	12s	10.244.0.26	minikube	<none>	<none>	<none>
signup-shard-validation-6-42hdv	0/1	Completed	0	12s	10.244.0.25	minikube	<none>	<none>	<none>
signup-shard-validation-5-7mcbv	0/1	Completed	0	13s	10.244.1.24	minikube-m02	<none>	<none>	<none>
signup-shard-validation-4-fjns5	0/1	Completed	0	13s	10.244.1.23	minikube-m02	<none>	<none>	<none>
signup-shard-validation-6-42hdv	0/1	Completed	0	13s	10.244.0.25	minikube	<none>	<none>	<none>
signup-shard-validation-5-7mcbv	0/1	Completed	0	13s	10.244.1.24	minikube-m02	<none>	<none>	<none>
signup-shard-validation-7-f4dqj	0/1	Completed	0	13s	10.244.0.26	minikube	<none>	<none>	<none>
signup-shard-validation-4-fjns5	0/1	Completed	0	13s	10.244.1.23	minikube-m02	<none>	<none>	<none>

3. Using the following command for **kubectl logs**:

```
for i in $(kubectl get pods -l job-name=signup-shard-validation \
-o jsonpath='{.items[*].metadata.name}'); do
  kubectl logs $i
done
```

```
ananya-kishore@ananya:~/da3408-assignment2/question_3$ for i in $(kubectl get pods -l job-name=signup-shard-validation -o jsonpath='{.items[*].metadata.name}'); do
  kubectl logs $i
done
shard_index=0 pod=signup-shard-validation-0-n8wp2 node=minikube-m02 total=50 invalid=10
shard_index=1 pod=signup-shard-validation-1-86mnn node=minikube-m02 total=50 invalid=16
shard_index=2 pod=signup-shard-validation-2-zcs85 node=minikube total=50 invalid=17
shard_index=3 pod=signup-shard-validation-3-zhtgn node=minikube total=50 invalid=13
shard_index=4 pod=signup-shard-validation-4-fjn5w node=minikube-m02 total=50 invalid=15
shard_index=5 pod=signup-shard-validation-5-7mc8v node=minikube-m02 total=50 invalid=21
shard_index=6 pod=signup-shard-validation-6-42hdv node=minikube total=50 invalid=13
shard_index=7 pod=signup-shard-validation-7-f4djg node=minikube total=50 invalid=21
```

Look at the last column in the above image (invalid) to get the invalid counts per shard.

The invalid counts are: shard 0: 10, shard 1: 16, shard 2: 17, shard 3: 13, shard 4: 15, shard 5: 21, shard 6: 13, shard 7: 21

Pod logs is used instead of a shared volume because here, the results are required to be collected via the Kubernetes API. Also, a shared volume would introduce a storage dependency across the 2 minikube nodes. However, the default minikube storage class does not make arbitrary pod-local data automatically shared between nodes

- The cluster would have $3 * 2 = 6$ CPUs, allowing us to increase parallelism from 4 to 6. With the new parallelism = 6, we can run 6 validation pods concurrently and make use of the additional CPU capacity. The 8 completions will still be processed in 2 sets (6 completions, then 2 completions).

Question 4

- At `question_4/deployment.yaml` and `question_4/service.yaml`
- Evidence:

```
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl get pods -l app=spam-api
NAME                                READY   STATUS    RESTARTS   AGE
spam-api-b7cddf48d-k9tdq            1/1     Running   0           31s
spam-api-b7cddf48d-n4jh6            1/1     Running   0           31s
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl delete pod spam-api-b7cddf48d-k9tdq
pod "spam-api-b7cddf48d-k9tdq" deleted from default namespace
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl get pods -l app=spam-api
NAME                                READY   STATUS    RESTARTS   AGE
spam-api-b7cddf48d-577hd            1/1     Running   0           10s
spam-api-b7cddf48d-n4jh6            1/1     Running   0           61s
```

The controller responsible for maintaining the number of pods is the **ReplicaSet controller** managed by the Deployment. When one pod is deleted, ReplicaSet detects that the actual number of pods (here, 1) is less than the desired number (here, 2) and automatically creates a replacement pod.

- Evidence:

```
ananya-kishore@ananya:~/da3408-assignment2/question_4$ minikube image load span-multistage:v2
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl set image deployment/span-api span-api=span-multistage:v2
deployment.apps/span-api image updated
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl rollout status deployment/span-api
Waiting for deployment "span-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "span-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "span-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "span-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api" rollout to finish: 1 old replicas are pending termination...
deployment "span-api" successfully rolled out
ananya-kishore@ananya:~/da3408-assignment2/question_4$ kubectl rollout history deployment/span-api
deployment.apps/span-api
REVISION   CHANGE-CAUSE
1          <none>
2          <none>
```

Deployment performs a rolling update by gradually replacing the pods running the old image (v1) with pods running the new image (v2), using the readiness probe to ensure the new pods are ready before receiving traffic. If this API were a job, it would run its pods to completion and stop, and changing the image would not cause a rolling replacement of continuously running API replicas.

- A **deployment** is designed for a long running serving workload such as this spam-detection API. It continuously maintains the desired number of replicas, replaces failed pods, and supports rolling updates while keeping the service available. A **job** is designed for a finite batch workload such as the hyperparameter search from the previous question: its pods perform work and terminate when the work is successfully completed, instead of remaining alive to serve requests.